Swiggy Delivery Time Prediction

Business Problem

Swiggy processes millions of food-delivery journeys across hundreds of cities every month. Accurate Estimated Time of Arrival (ETA) at order placement and during delivery is core to the customer experience — it affects conversion, cancellations, ratings, and retention. Today's ETA accuracy suffers from many sources of uncertainty: highly variable traffic patterns, weather events, restaurant preparation variability, rider behaviour and skill, order batching/multiple deliveries, and city-specific operational constraints. Inaccurate ETAs cause:

- Reduced customer trust and lower repeat orders.
- Increased cancellations and refund costs.
- Higher support volume and operational interventions.
- Sub-optimal rider allocation and increased rider idle or overtime costs.

Problem Statement

Swiggy's objective is to predict the delivery time (minutes) per order at the moment of order placement (and update it dynamically), with business-grade accuracy and uncertainty estimates so the platform can

(a) show reliable ETAs to customers

(b) optimize rider allocation.

Build a production-ready, scalable Machine Learning system that predicts per-order delivery time (in minutes) using real-time and historical features (order, rider, restaurant, geospatial, traffic, weather, and temporal signals). The system must produce:

Dataset:-https://drive.google.com/file/d/1ZPVuwIDwRMcmKHCgJY8KU9YYFtTbSTIz/view?usp=drive_link

Dataset Overview

Rider_id:- Unique identifier for each delivery partner.

Age **:**- Age of the delivery partner.

Ratings **:**- Average customer rating of the rider.

vehicle_condition:-Condition of the rider's vehicle.(0 => poor ,1=>Average, 2=>Good)

Type_of_vehicle :-Vehicle used for delivery (motorcycle, scooter, etc.).

Type_of_order :-Type of food ordered (snack, meal, drinks, buffet).

multiple_deliveries:-Number of deliveries assigned to the rider in a single trip.

pickup_time_minutes:-Time taken by the restaurant to prepare the order.

restaurant_longitude:-Geographic coordinates of the restaurant.

delivery_longitude:- Geographic coordinates of the customer's delivery location.

Is_weekend **:-**Indicates whether the order was placed on a weekend.

Order_time_hour **:**-Hour when the order was placed (0–23).

order_time_of_day:- Time bucket (morning, afternoon, evening, night).

Weather **:**-Weather condition during delivery (sunny, cloudy, fog, stormy, sandstorms).

Traffic **:**-Traffic condition during delivery.

Festival :-Indicates whether the order was placed on a festival day.

City_type **:**-Classification of city (urban, metropolitan).

City_name **:**-Name/code of the city (BANG, HYD, CHEN, etc.).

Time_taken **:**- Actual time taken (in minutes) to complete the delivery.

Distance :-Distance (in km) between restaurant and delivery location.

Order_date **:**-Date when the order was placed.

Order_day **:**-Day of the month.

Order_month **:**-Month of the order.

order_day_of_week:-Day of the week (Monday–Sunday).

Project Instructions

Step 1: **Understand the CRISP-ML(Q) Methodology**

- **Business & Data Understanding**: Define the problem clearly (Classification or Regression). Identify the target variable and features.
- **Data Preparation**: Clean missing values, handle categorical variables, scale numerical features, and engineer new features if needed.
- **Modeling**: Build multiple machine learning models.
- **Evaluation**: Compare models using error metrics (RMSE, MAE) and R^2 score.
- **Deployment**: Deploy the best model for real-world use.

Step 2: Data Preprocessing

- Handle missing values.
- Encode categorical variables.
- Scale numerical features.
- Apply the Transformation if required.
- Feature Selection
- Split data into **train/test sets**.

Note:- Apply the Preprocessing using Pipeline and without using Pipeline.

Step 3: Model Building

- Train multiple Machine Learnings models/Algorithms:
- Use **cross-validation** to evaluate performance consistently.

Step 4: Hyperparameter Tuning

- Apply **Hyperparemeter Tuning** to optimize parameters.
- Example:
    - Random Forest → n_estimators, max_depth, min_samples_split.
- Select the best parameters based on **lowest error** and **highest R²**.

Step 5: Model Selection

- Compare models using metrics:
    - **Error metrics**: RMSE, MSE.
    - **Goodness of fit**: R² score.
    -
- Choose the model with **lowest error** and **highest R²**.

Step 6: Deployment

- **Streamlit**:
    - Build a simple UI where users can input data and get predictions.
    - Example: st.text_input(), st.button(), st.write(prediction).
- **Hugging Face Spaces**:
- Push your Streamlit app to Hugging Face Spaces for free hosting.
- Ensure requirements.txt includes all dependencies (scikit-learn, pandas, streamlit).